PyLab

Python for Life Scientists — a hands-on companion

EMBL-EBI · Intro to Python Mahfouz Shehu & Sandy Rogers

GETTING STARTED

How PyLab works

- **Code in your browser**

Powered by Pyodide — real Python, nothing to install.

- **30 challenges, 6 tiers**

Earn XP, unlock tiers, climb the live leaderboard.

- **Every challenge follows one rhythm**

R reminder → Python equivalent → your task → auto-graded tests.

- **Learn by fixing, too**

A Bug Hunt tier trains you to spot real Python gotchas.

PLAY HERE

`intro-to-python-game-production.up.railway.app/game`

Tip: sign in with a nickname so your XP shows on the leaderboard.

BEFORE YOU START

The R → Python mindset

You already think like a programmer. These are the shifts that trip up R users most:

R	Python	Why it matters
`x <- 5`	`x = 5`	Assignment with =
`x[1]  (1-based)`	`x[0]  (0-based)`	The #1 gotcha
`NULL / NA`	`None / NaN`	Use `is None` to test
`c(), list()`	`list  tuple  dict  set`	More built-in structures
`sapply(x, f)`	`[f(i) for i in x]`	List comprehensions
`data.frame`	`pandas.DataFrame`	Same idea, new API

TIER 1

Nucleotide

Python fundamentals

Challenges 1–5 · 100–150 XP

TIER 1 · NUCLEOTIDE

Output, indexing & types

- **print() auto-newlines**
No \n needed, unlike R's cat().
- **0-based & end-exclusive**
seq[0] is the first base; seq[0:3] = ATG.
- **Negative indexing**
seq[-1] is the last base — no len() needed.
- **type() & None**
int/float/str/bool; use is None, not == None.

R

```r
cat("Hello\n")
seq <- "ATGCGT"
substr(seq,1,1) # "A"
substr(seq,1,3) # "ATG"
is.na(x); is.null(x)
```

Python

```python
print("Hello")
seq = "ATGCGT"
seq[0]    # "A"
seq[0:3]  # "ATG"
seq[-1]   # last base
x is None
```

String surgery: .upper() .replace() .strip() .split() — methods chain off the value itself.

TIER 2

Amino Acid

Data structures & decisions

Challenges 6–10 · 200–250 XP

TIER 2 · AMINO ACID

Lists, dicts, sets, tuples

- **list**

Ordered, mutable: genes = ['BRCA1', ...]

- **dict**

Key→value (R named list): codon['ATG']

- **set**

Unique items + & | - for overlaps

- **tuple**

Immutable coords: (chrom, start, end)

R

```r
genes <- c("A","B")
m <- list(ATG="Met")
m[["ATG"]]
union(a, b)
intersect(a, b)
```

Python

```python
genes = ["A", "B"]
m = {"ATG": "Met"}
m["ATG"]
m.get("XXX")  # None
a | b   # union
a & b   # intersect
```

Decisions: if / elif / else, the in operator, and f-strings — see the Variant Classifier (challenge 10).

TIER 3

Organelle

Control flow & functions

Challenges 11–15 · 350–400 XP

TIER 3 · ORGANELLE

Loops, comprehensions & functions

- **for + enumerate()**

Loop with an index, cleanly.

- **List comprehension**

The Pythonic sapply().

- **def with defaults**

normalize(x, method='zscore')

- **try / except**

Python's tryCatch().

R

```r
for (g in genes) {
  cat(g)
}
sapply(x, sqrt)
f <- function(x) x^2
tryCatch(..., error=)
```

Python

```python
for i, g in \
    enumerate(genes):
    print(i, g)
[sqrt(v) for v in x]
def f(x): return x**2
try: ...
except ValueError: ...
```

f-strings format inline: f"{gene}: {value:.2f}" ≈ R's glue() / sprintf().

TIER 4

Organism

Working with real data

Challenges 16–19 · 500 XP

TIER 4 · ORGANISM

Files, grouping, stats & regex

- **csv.DictReader**

Read rows as dicts; cast types yourself.

- **defaultdict**

Group-by from scratch (the dplyr idea).

- **statistics / scipy**

Means, stdev, Welch's t-test.

- **re module**

Pattern-match motifs & IDs.

R

```
read.csv("d.csv")
aggregate(x~g,
FUN=mean)
t.test(a, b)
grepl("p[0-9]+", x)
```

Python

```
csv.DictReader(f)
d = defaultdict(list)
scipy.stats.\
  ttest_ind(a, b)
re.findall("p[0-9]+",s)
```

These mirror pandas — once you see the manual version, df.groupby() makes sense.

TIER 5

Ecosystem

Putting it all together

Challenges 20–22 · 750–1000 XP

TIER 5 · ECOSYSTEM

Parsers, k-mers & a pipeline

- **FASTA parser**
Multi-line records via a generator.
- **Counter**
Tally k-mers in one line.
- **dict comprehension**
{k: v for ...} builds maps fast.
- **Mini pipeline**
DESeq2-style: integrate everything.

R

```
readLines("x.fasta")
table(kmers)
# manual loops to
# join + summarise
Biostrings / DESeq2
```

Python

```
for h, seq in \
    parse(path):
Counter(kmers)
{k: f(k) for k in ks}
# biopython preview
```

Challenge 22 chains reading → filtering → joining → summarising — a real analysis in pure Python.

TIER 6

Bug Hunt

Spot & fix real Python gotchas

Challenges 23–30 · 300–500 XP

TIER 6 · BUG HUNT

Eight gotchas to internalise

Bug	The fix
Off-by-one from 1-based habits	Remember seq[0]; slices exclude the end
Mutating a shared list	Copy with list(x) or x.copy()
"n=" + 5 → TypeError	Cast: "n=" + str(5) or use an f-string
dict[missing] → KeyError	Use dict.get(key, default)
7 / 2 is 3.5, not 3	Use // for integer (floor) division
UnboundLocalError in a function	Pass values in/out; avoid hidden global writes
def f(x, acc=[]) reuses the list	Default to None, create inside the function
if x == None	Use if x is None (PEP 8)

Now go play 🐍

- Work tier by tier — each builds on the last.
- Stuck? Use the hints, read the test feedback, iterate.
- Compare every answer to how you'd do it in R.
- Next steps beyond the game: pandas · seaborn · Biopython.

Leaderboard updates live — challenge a colleague!